

CSA0404 OPERATING SYSTEM CO3 AT1

EXPERIMENTS

NAME: ANGEL DEEPIKA M

REG NO: 192521272

1.Design and implement a program to simulate the translation of a logical address into a physical address using base and limit registers. The program should accept logical address as input, check for validity, and compute the corresponding physical address. Display appropriate messages for valid and invalid addresses.

Logical Address Translation Using Base and Limit Registers

Aim

To simulate the translation of a logical address into a physical address using base and limit registers.

EXPLANATION

The base register contains the starting physical address and the limit register specifies the size of the allocated memory.

Physical Address = Base + Logical Address

Valid if: $0 \leq \text{Logical Address} < \text{Limit}$

Algorithm

1. Read base, limit and logical address.
2. Check whether the logical address is within the limit.
3. If valid, calculate the physical address.
4. Otherwise, display an invalid address message.

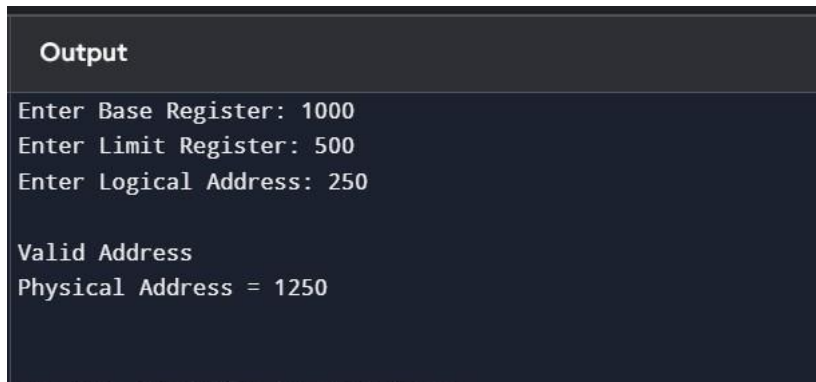
C Program

```
#include <stdio.h> int main() {    int base,  
limit, logical, physical;    printf("Enter  
Base Register: ");    scanf("%d", &base);  
printf("Enter Limit Register: ");
```

```

scanf("%d", &limit);    printf("Enter
Logical
Address: ");    scanf("%d",
&logical);    if
(logical >= 0 && logical < limit)
    {    physical = base + logical;    printf("\nValid
Address");    printf("\nPhysical
Address = %d\n", physical);
    }    else
    {    printf("\nInvalid
Address");    printf("\nAccess
Denied\n");
    }
return 0;
}

```



```

Output
Enter Base Register: 1000
Enter Limit Register: 500
Enter Logical Address: 250

Valid Address
Physical Address = 1250

```

Result

Thus, logical address translation using base and limit registers was successfully implemented.

2. Write a program to demonstrate the paging memory management scheme. The program should divide memory into fixed-size pages and frames, accept page number and offset as input, and compute the corresponding physical address using a page table.

Paging Memory Management Scheme

Aim

To demonstrate paging memory management by mapping a page number and offset to a physical address using a page table.

EXPLANATION

In paging, logical memory is divided into pages and physical memory into frames.

Physical Address = (Frame Number $\times$ Page Size) + Offset

Algorithm

1. Read page size and number of pages.
2. Enter the frame number for each page.
3. Read page number and offset.
4. Find the corresponding frame from the page table.
5. Calculate the physical address.
6. Display the result.

C Program

```
#include <stdio.h> int main() {    int
pageTable[10];    int pageSize, pages;
int page, offset, frame, physical;

    int i;

    printf("Enter Page Size: ");    scanf("%d",
&pageSize);    printf("Enter
Number of Pages: ");    scanf("%d",
&pages);    printf("\nEnter Page Table:\n");
for (i = 0; i < pages; i++)
    {        printf("Page %d -> Frame:
", i);        scanf("%d",
&pageTable[i]);
```

```

    }

    printf("\nEnter Page Number: ");    scanf("%d", &page);

    printf("Enter Offset: ");    scanf("%d", &offset);    if (page >= 0 && page
< pages && offset >= 0 && offset < pageSize)

    {

        frame = pageTable[page];    physical =
(frame * pageSize) + offset;    printf("\nFrame
Number = %d", frame);    printf("\nPhysical
Address = %d\n", physical);

    }    else

    {

        printf("\nInvalid Page Number or Offset.\n");

    }

return 0;

}

```

```

Output
Clear

^ Enter Page Size: 100
Enter Number of Pages: 4

Enter Page Table:
Page 0 -> Frame: 3
Page 1 -> Frame: 1
Page 2 -> Frame: 4
Page 3 -> Frame: 2

Enter Page Number: 2
Enter Offset: 25

Frame Number = 4
Physical Address = 425

=== Code Execution Successful ===

```

Result

Thus, the paging memory management scheme was successfully implemented.

3. Develop a program to simulate a page table. The program should allow the user to input page table entries and logical addresses, then map them to physical addresses. Clearly show how page numbers are translated into frame numbers.

Page Table Simulation

Aim

To simulate a page table and translate a logical address into a physical address.

EXPLANATION

A logical address consists of a page number and offset. The page table maps the page number to a frame number.

Page Number → Page Table → Frame Number

Physical Address = (Frame × Page Size) + Offset

Algorithm

1. Read page size and page table entries.
2. Read the logical address.
3. Calculate page number and offset.
4. Find the corresponding frame number.
5. Calculate the physical address.
6. Display the translation.

C Program

```
#include <stdio.h> int main() {    int
pageTable[10];    int pageSize, pages,
logical;    int page, offset, frame,
physical;

    int i;

    printf("Enter Page Size: ");    scanf("%d",
&pageSize);    printf("Enter Number of Pages:
");    scanf("%d", &pages);    printf("\nEnter
Page Table:\n");    for
(i = 0; i < pages; i++)
    {        printf("Page %d -> Frame:
```

```

", i);    scanf("%d",
&pageTable[i]);
    }

    printf("\nEnter Logical Address: ");
    scanf("%d", &logical);    page = logical
/ pageSize;    offset = logical
% pageSize;    if (page >= 0 && page
< pages)
    {
        frame = pageTable[page];
        physical = (frame * pageSize) + offset;
        printf("\nPage Number = %d", page);
        printf("\nOffset = %d", offset);
        printf("\nFrame Number = %d", frame);
        printf("\nPhysical Address = %d\n", physical);
    }    else
    {
        printf("\nInvalid Logical Address.\n");
    }

return 0; }

```

```

Output
Enter Page Size: 100
Enter Number of Pages: 5

Enter Page Table:
Page 0 -> Frame: 2
Page 1 -> Frame: 5
Page 2 -> Frame: 1
Page 3 -> Frame: 4
Page 4 -> Frame: 3

Enter Logical Address: 245

Page Number = 2
Offset = 45
Frame Number = 1
Physical Address = 145

=== Code Execution Successful ===

```

Result

Thus, the page table was successfully simulated and the logical address was mapped to its physical address.

4. Implement a program to simulate the working of a Translation Lookaside Buffer (TLB). The program should demonstrate how TLB improves memory access time by calculating the effective memory access time with and without TLB.

Translation Lookaside Buffer (TLB)

Aim

To simulate the working of TLB and calculate the effective memory access time with and without TLB.

EXPLANATION

TLB is a fast cache that stores recently used page-to-frame mappings. A TLB hit reduces the time required for address translation.

Miss Ratio = 1 - Hit Ratio

EAT without TLB = $2 \times \text{Memory Time}$

EAT with TLB = TLB Time + (Hit Ratio $\times$ Memory Time) + (Miss Ratio $\times 2 \times$ Memory Time)

Algorithm

1. Read memory access time and TLB lookup time.
2. Read TLB hit ratio.
3. Calculate miss ratio.
4. Calculate EAT with and without TLB.
5. Display and compare both values.

C Program

```
#include <stdio.h> int main() {    float memory,  
tlb, hit, miss;    float withoutTLB, withTLB;  
printf("Enter  
Memory Access Time: ");    scanf("%f",
```

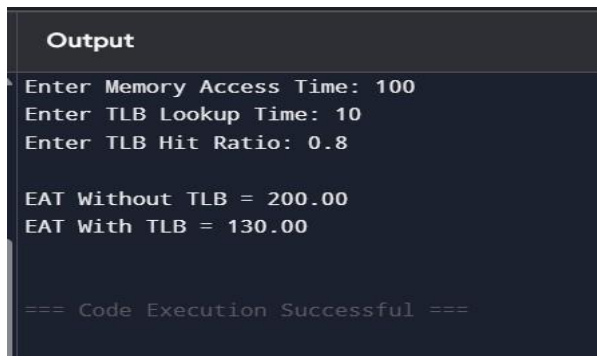
```

&memory);    printf("Enter TLB Lookup
Time: ");    scanf("%f", &tlb);    printf("Enter
TLB Hit Ratio: ");    scanf("%f", &hit);    if
(hit < 0 || hit > 1)
    {        printf("Invalid Hit
Ratio\n");    return 0;

    }    miss = 1 - hit;    withoutTLB = 2 * memory;    withTLB
= tlb + (hit * memory) + (miss * 2 * memory);    printf("\nEAT
Without TLB = %.2f", withoutTLB);    printf("\nEAT With TLB
= %.2f\n", withTLB);

    return 0;
}

```



```

Output
Enter Memory Access Time: 100
Enter TLB Lookup Time: 10
Enter TLB Hit Ratio: 0.8

EAT Without TLB = 200.00
EAT With TLB = 130.00

=== Code Execution Successful ===

```

Result

Thus, the TLB mechanism was successfully simulated and its effect on memory access time was calculated.

5. Write a program to illustrate memory protection using base and limit registers. The program should verify whether a given logical address lies within the valid range and prevent unauthorized access.

Memory Protection Using Base and Limit Registers

Aim

To implement memory protection using base and limit registers and prevent unauthorized memory access.

EXPLANATION

The base register specifies the starting address and the limit register specifies the size of the allocated memory.

Valid if: $0 \leq \text{Logical Address} < \text{Limit}$

Physical Address = Base + Logical Address

An address outside the limit is rejected.

Algorithm

1. Read base and limit values.
2. Read the logical address.
3. Check whether it is within the valid range.
4. If valid, calculate the physical address and grant access.
5. Otherwise, deny access.

C Program

```
#include <stdio.h> int main() {    int base,
limit, logical, physical;    printf("Enter Base
Register: ");    scanf("%d", &base);
printf("Enter Limit Register: ");
scanf("%d", &limit);    printf("Enter
Logical
Address: ");    scanf("%d", &logical);    if
(logical >= 0 && logical < limit)
    {        physical = base + logical;
printf("\nAccess Granted");        printf("\nPhysical
Address = %d\n", physical);    }    else
{        printf("\nAccess Denied");
printf("\nUnauthorized Memory Access\n");
    }
```

```
return 0;
```

```
}
```

```
Output
Enter Base Register: 2000
Enter Limit Register: 5000
Enter Logical Address: 200

Access Granted
Physical Address = 2200

=== Code Execution Successful ===
```

Result

Thus, memory protection using base and limit registers was successfully implemented, and unauthorized memory access was prevented.